iChart-lite: Accessible Charts for the Visually Impaired using Lightweight Extraction Algorithm

Gihun Nam[O1] Danu Kim[2]

[1]College of Computing and Informatics, Sungkyunkwan University, Suwon 16419, Republic of Korea
[2]Korea International School, Jeju Campus, Jeju-do 63644, Republic of Korea
sitd2813@gmail.com, dukim27@kis.ac

Abstract

People use charts for visualizing data, but they pose an accessibility issue for visually impaired people. This issue has become an even more challenge with the prevalence of online courses and meetings where charts are served within real-time video. In this paper, we introduce iChart-lite, a real-time assistive system that addresses this issue by automatically extracting data from bar charts and generating textual and auditorial descriptions. Our system utilizes real-time object detection and heuristics, enabling it to be lightweight enough to run on portable devices. In addition, we tested our model with real-world data to enhance system robustness. Our quantitative and qualitative analysis demonstrates that iChart-lite is a viable solution for visually impaired users to understand bar charts. The code, associated data, and demo video can be accessed at https://github.com/iChart-lite/iChart-lite.

1. Introduction

Charts are a convenient tool to visualize quantitative data. Nevertheless, they are highly inaccessible to the visually impaired because most charts are served solely in image format that requires a vision process. The significance of this issue has been further growing since the rise of online courses and virtual meetings, where charts are presented predominantly through real-time videos. A few studies tackled this issue by automatically extracting data from chart images. ReVision [1], one of the most well-known research in this field, tried to solve the problem with the SVMs classifier and classical image processing techniques. Moreover, in the case of ChartSense [2], they added user interaction and response to the previous approach to improve the low extraction accuracy. ChartOCR [3], on the other hand, adopted a neural network approach, using a key point detection model to create a faster and more accurate system.

Despite the satisfactory performance of the previous works, they share common limitations: Firstly, these works require high-performance and high-cost devices (e.g., NVIDIA Tesla GPGPU, Intel Xeon), which are not easily accessible by the public. Even if one has such a device, these solutions are not portable, rendering them impractical for general use cases. Secondly, they rely on artificially generated charts with a well-structured layout, which often fails to capture the variety of real-world charts. Lastly, real-time videos were not the domain of their research. As videos contain artifacts and objects unrelated to charts, previous systems can report higher error rates by recognizing a wrong object. Due to these limitations, applying these systems to the everyday lives of the visually impaired is challenging.

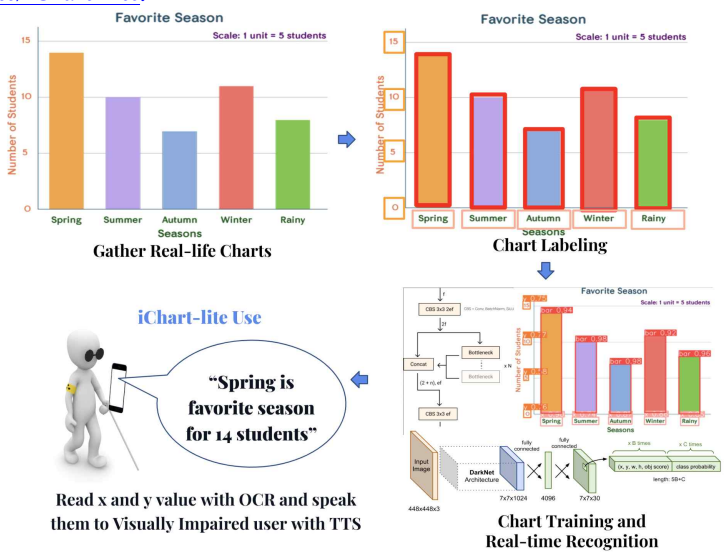

Figure 1 Overview of the iChart-lite system

Thus, we propose iChart-lite, an assistive system that helps the visually impaired by automatically extracting bar chart data and generating textual descriptions, which are delivered to the user using Text-to-Speech (TTS). The iChart-lite uses a real-time object detection technique and heuristics to overcome the limitations of previous works. Moreover, we created a custom dataset that consists of real-world bar chart images from the internet to improve the validity of our system further. Lastly, iChart-lite is lightweight enough to be run on portable devices like laptops while maintaining good performance and accuracy against real-world data.

2. iChart-lite System

Figure 1 shows that our system comprises three major parts: object detection, data extraction, and textual and

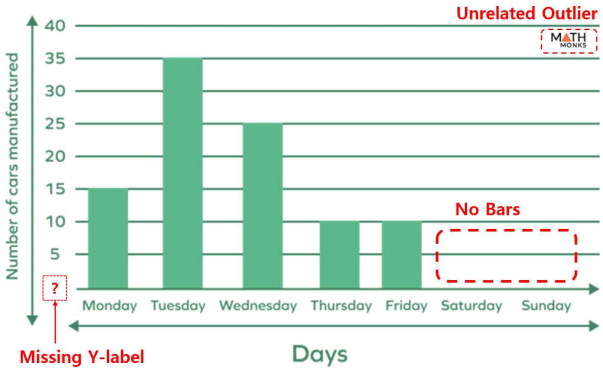

Figure 2 Real-life chart with abnormalities

auditorial description generation. Object detector outputs bounding boxes of the bar, X-label, and Y-label from a chart image, and these bounding boxes are fed to the data extractor, which retrieves the value from each bounding box using OCR and heuristics. The extracted values are transformed into textual descriptions and processed by the TTS, generating user-facing audio.

2.1 Gathering Dataset

Real-world chart data often contains various abnormalities (in Fig. 2). Some charts omit the base Y label and have a suffix or prefix (i.e., $, K) on their Y labels. They also come in varying sizes and resolutions. To better represent these characteristics of real-world data, we collected 68 bar chart images of varying structures from diverse web sources. We trained our object detection model against this custom dataset. Each image contains labels of three classes: bar, X-label, and Y-label. Each class's total number of instances is 374 for bar, 378 for X-label, and 468 for Y-label.

2.2 Detecting Objects and Removing Outliers

Locating the components in a chart is a challenging task. Traditional methods used two-stage approaches to detect texts and bars separately. However, in this study, we tackled the problem in single-step component detection, which leads to lightweight and real-time response by employing a fast and robust object detection model. To this end, we exploited Ultralytics YOLOv8, the state-of-the-art object detection model.

Object detection occasionally locates unrelated outliers that should not be recognized; it incorrectly recognizes legends, titles, or unrelated numbers as X or Y labels. We apply the IQR method to filter out the incorrectly detected objects to mitigate this issue. IQR is applied to X-label and bar classes. Let the bound box of an object be $[x_t,y_t,x_b,y_b]$, which means its top-left X and Y coordinates and its bottom-right X and Y coordinates consecutively. Then, for Q1, Q3, and IQR calculations in Eq. 1, we used the mean of two y-coordinates, $(y_t+y_b)/2$, for bars and the bottom y-coordinate, y_b, for X-labels. OFFSET parameter was also used to prevent correctly detected objects from being removed.

$$L = Q_1 - 1.5 \times IQR - OFFSET$$
$$U = Q_3 + 1.5 \times IQR - OFFSET \quad (1)$$

If an object is outside of [L, U], then it is an outlier.

2.3 Extracting Label Texts and Estimating Bar Height

We utilized Tesseract OCR [4] to extract the texts from the detected X and Y-label; regular expression was used to extract value and suffix/prefix from the raw texts for the Y-label texts. We used heuristics in Eq. 2 to estimate the height of the bars. Given a bounding box of a bar, $[x_t,y_t,x_b,y_b]$, and the X, Y coordinates and a value of two Y-labels be $[m_x,m_y,m_v]$ and $[n_x,n_y,n_v]$, then we estimate the height value, h_e, as:

$$h_e = \frac{n_v - m_v}{n_y - m_y} \times (y_t - m_y) + m_v \quad (2)$$

2.4 Generating Chart Description

Extracted components from 2.2 are then converted into the textual description. Firstly, the missing bars and x-labels must be examined because some charts had omitted bars with 0 values, and the object detector could not recognize this. Further, there might be a detection error, such as not recognizing the correct bar or X-label. We fixed these errors by heuristic-based post-processing, e.g., we assigned a default value (0) for the case of an X-label without a corresponding bar. After that, a textual description is generated with a text substitution using the description template. Finally, the generated text is translated into a user-facing auditory description by Google TTS [5].

3. Experimental Results

We used YOLO's pre-trained weights as a base point and additionally trained the model on the custom data. The dataset was split into a ratio of 8:2 and 9:1 for training-validation set pairs. The training was conducted on Google Colaboratory using a T4 GPU instance with a batch size of 16 for 30 epochs. The OFFSET was set to be 7 because it provided best performance.

3.1 Quantitative Analysis: Running time and mAP

We first evaluated our system regarding time efficiency. The training took 133.2 and 115.2 seconds for 8:2 and 9:1 validation, respectively. The test was performed on two environments to compare the efficiency depending on devices: a Google Colab with a T4 GPU and a low-power laptop (Surface Pro 8) with an Intel i5-1135G7 CPU. In Table 1, on a Colab, the detection and extraction process took 0.02 and 0.05 seconds, respectively; on the laptop, 0.25 and 0.04 seconds. The difference in total duration between the

two devices was only 0.22 seconds, showing that our system is not constrained to high-end machines.

Task	Time in Colab/GPU	Time in Surface/i5
Detection	0.02	0.25
Extraction	0.05	0.04

Table 1. Average running time in seconds

We evaluated our system's accuracy in recognizing chart components. We see from Table 2 that the overall mAP metric shows high precision for recognizing the bar, X-label, and Y-label; overall performance in 9:1 validation shows 99.3% of mAP.

Class	mAP (8:2 ratio)	mAP (9:1 ratio)
Bar	0.955	0.995
X-label	0.995	0.995
Y-label	0.863	0.988

Table 2. Accuracy comparison according to validation size

For quantitative evaluation of data extraction quality, we employ two different measures: the error rate of the X-label and bar height estimation. The error rate of the X-label, E_X, was calculated with the Levenshtein distance [6] between the ground truth string, S_g, and the predicted string, S_p, divided by the length of S_g, as shown in Eq. 3(left). Bar's error rate, E_h, was computed using Eq. 3(right); the difference between the ground truth, h_g, and estimated height, h_e, was normalized.

$$E_X = \frac{Levenshtein(s_g, s_p)}{max(s_g, s_p)}, \quad E_h = \frac{|h_g - h_e|}{h_g} \qquad (3)$$

Analysis	X-label Extract	Height Estimate
Error rate	0.042	0.019

Table 3. Error rate of X-label and bar height estimation

Table 3 lists the extraction errors. Both reported low error rates: 4.2% for X-label and 1.9% for bar height. The X-label error mainly results from the OCR engine struggling to recognize single-digit labels. However, this problem can be easily solved by further tuning the OCR engine.

3.2 Qualitative Analysis: Subjective Evaluation

We perform a subjective evaluation to examine further the iChart-lite output's quality. We surveyed six participants, asking about the results' perfectness level, as shown in Fig. 3. The overall performance was evaluated using a 5-point Likert scale. Table 4 shows that our system scored 4.35 out of 5.0 on average; 86% of responses reported over 4.0 scores.

Score	1.0	2.0	3.0	4.0	5.0
Percent	3.7%	3.0%	7.3%	26.9%	59.1%

Table 4. Score distribution in users' subjective evaluation

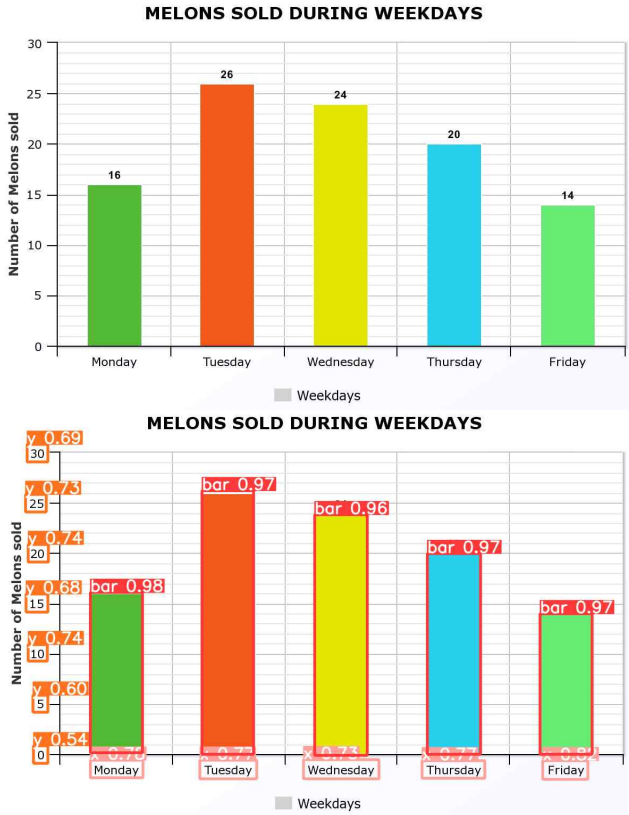

Figure 3. A detected chart and its output generated for 'Melons Sold During Weekdays': "Monday is 16.11, Tudesday is 26.24, Wednesday is 23.90, and so on."

4. Conclusion

We presented iChart-lite, a chart narrative assistant for the visually impaired that performs well on lightweight devices. The quantitative and qualitative analysis showed that the iChart-lite is an effective tool to help the visually impaired understand graphical bar charts. To make this study more effective, we plan to develop a more sophisticated algorithm to handle other charts and apply efficient OCR methods to our method because OCR is the time-consuming part of iChart-lite.

References

[1] M. Savva, "ReVision: Automated Classification, Analysis and Redesign of Chart Images," 24th ACM symposium on User interface software and technology, pp.393-402, 2011.

[2] D. Jung et al., "ChartSense: Interactive Data Extraction from Chart Images," CHI conference on human factors in computing systems, 2017.

[3] J. Luo et al., "ChartOCR: Data Extraction from Charts Images via a Deep Hybrid Framework," IEEE/CVF WACV, pp.1917-1925, 2021.

[4] Tesseract OCR, https://github.com/tesseract-ocr/tesseract

[5] Google Text-to-Speech, https://gtts.readthedocs.io

[6] V. Levenshtein, "Binary Codes Capable of Correcting Deletions, Insertions, and Reversals," Soviet physics doklady, Vol. 10, No. 8, pp. 707-710, 1966.